RAPPORT DE PROJET · M2 DEVOPS · RNCP 40166

SouverainRecrut

Chaîne CI/CD du Backend

Du `git push` à la production : tests, qualité, image signée par son commit, provisionnement et déploiement approuvé, avec retour arrière éprouvé.

330	100 %	PASSED	0
tests automatisés	de couverture	Quality Gate SonarQube	vulnérabilité bloquante

GitHub — Jenkins — SonarQube — Trivy — Docker Hub — GitLab CI — VPS

Martin BELLOT

M2 · SUP DE VINCI Rennes · novembre 2026

Dépôt	github.com/MartinBellot/souverainrecrut · Dev
Déploiement	gitlab.com/martin.bellot.off/souverainrecrut
Image	hub.docker.com/r/martinbellot/souverainrecrut-backend
API	148.230.115.48:4000/v1/info · Swagger

SOMMAIRE

Contenu du rapport

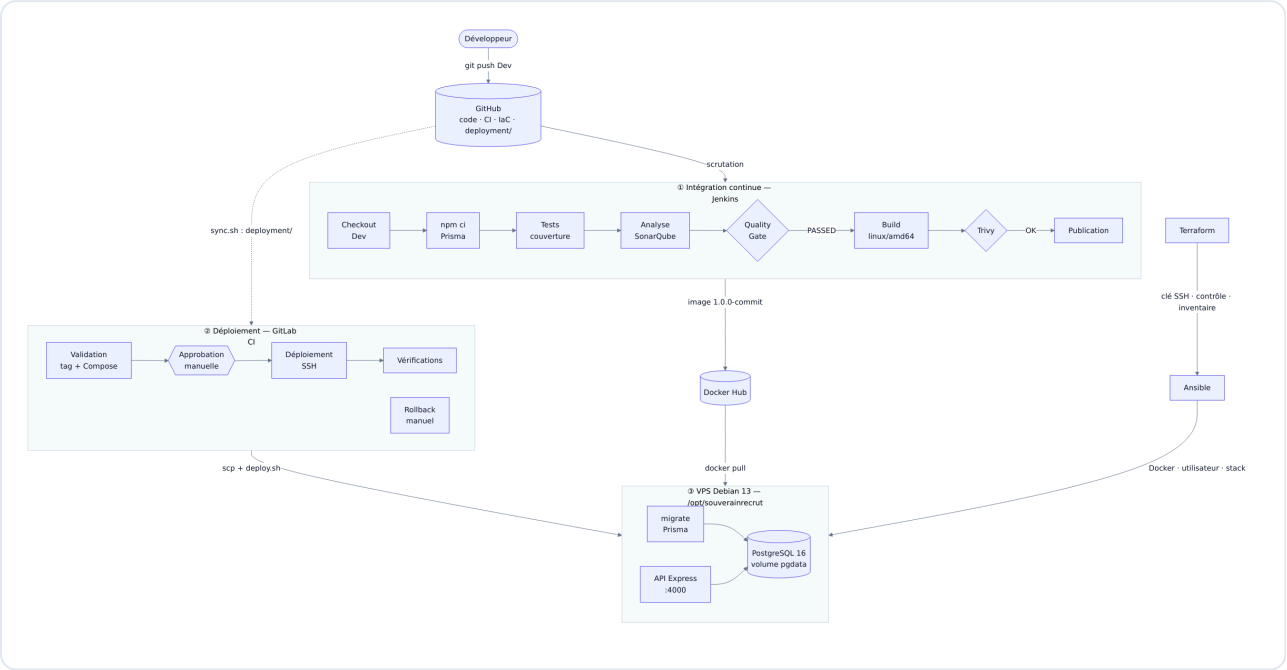
01	Synthèse	3
02	Architecture	4
03	Qualité : tests et SonarQube	6
04	Intégration continue : Jenkins	9
05	Image Docker et sécurité	11
06	Infrastructure : Terraform et Ansible	13
07	Déploiement : Docker Compose et GitLab CI	15
08	Vérification, rollback et données	19
09	Gestion des secrets	23
10	Conformité aux exigences	24
11	Défauts détectés et corrigés	25
12	Limites et pistes	26
13	Procédures de référence	27

01 Synthèse

Une modification du Backend poussée sur `Dev` est **testée, analysée, construite, scannée et publiée** automatiquement par Jenkins. Elle est ensuite **déployée sur approbation** par GitLab CI sur un serveur provisionné par Terraform et Ansible.

INDICATEUR	RÉSULTAT
Tests	**330** (244 unitaires · 64 intégration · 22 E2E), tous verts
Couverture	**100 %** (lignes, branches, fonctions)
SonarQube	Quality Gate **PASSED** · 0 bug · 0 vulnérabilité · 0 code smell · 0 % duplication
Trivy	**0** vulnérabilité bloquante · 0 secret dans l'image
Image	`martinbellot/souverainrecrut-backend:<version>-<commit>` (linux/amd64)
Production	API + PostgreSQL sur le VPS, déployés et vérifiés par GitLab CI
Continuité	Rollback testé (automatique, par job GitLab, et restauration de la base)
Secrets	0 dans le dépôt (13 commits analysés), 0 dans les journaux CI

02 Architecture



OUTIL	RÔLE	OÙ IL TOURNE
GitHub	Source unique : code, tests, `Jenkinsfile`, Terraform, Ansible, `deployment/`	SaaS
Jenkins	Intégration continue : tests → SonarQube → image → Trivy → Docker Hub	Conteneur local (`docker-compose.ci.yml`)
SonarQube	Qualité, sécurité et couverture, Quality Gate bloquant	Conteneur local
Trivy	Scan des vulnérabilités et des secrets de l'image	Dans l'image Jenkins
Docker Hub	Registre des images versionnées	SaaS
Terraform	Clé de déploiement, contrôle du serveur, inventaire Ansible	Poste d'administration
Ansible	Configuration du serveur et installation initiale de la stack	Poste d'administration → VPS
GitLab CI	Déploiement continu avec approbation, vérification et rollback	SaaS (runners GitLab)
Docker Compose	Orchestration API + PostgreSQL sur le serveur	VPS

CHOIX STRUCTURANTS

- **Séparer CI et CD** : Jenkins *produit* une image vérifiée, GitLab la *déploie*. Le serveur ne reconstruit jamais l'image : ce qui est déployé est exactement ce qui a été testé et scanné.
- **Une source unique pour le déploiement** : le dossier `deployment/` est utilisé à la fois par Ansible (installation) et par GitLab (mises à jour). Le projet GitLab en est une copie générée par `gitlab-deploy/sync.sh`.
- **Tout en code** : Jenkins (JCasC), Quality Gate (script), infrastructure (Terraform, Ansible), déploiement (Compose, scripts). Une procédure manuelle n'est jamais nécessaire.
- **Pas de VMware** : le déploiement cible un VPS existant (Debian 13), qui héberge aussi d'autres services. Toutes les opérations ont donc été conçues pour être **non intrusives** (stack, réseau et volume isolés, aucun redémarrage imposé).

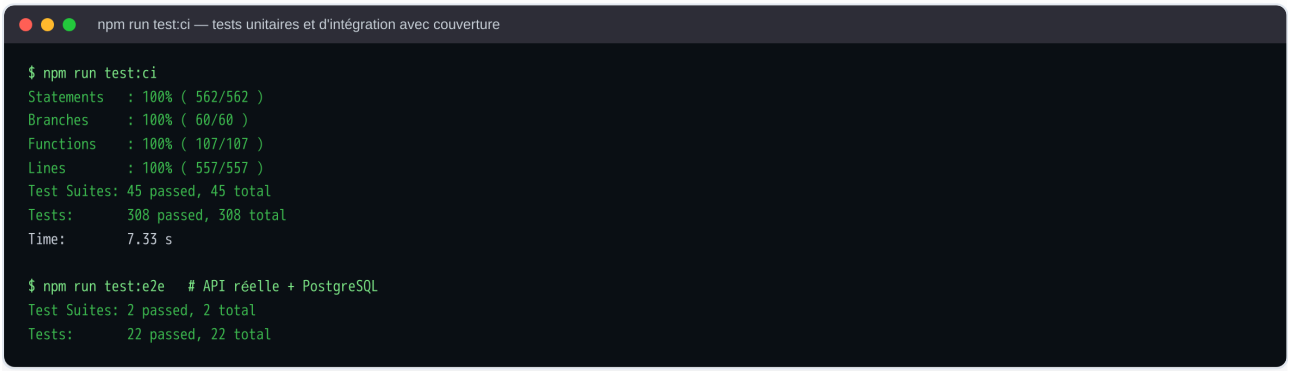
```
souverainrecrut/
├── backend/            API (Express, Prisma), tests, Dockerfile, sonar-
project.properties
├── Jenkinsfile         pipeline CI
├── ci/                 image Jenkins (JCasC, plugins), script du Quality Gate
├── terraform/          main.tf, variables.tf, outputs.tf, providers.tf
├── ansible/            inventory.ini, playbook.yml, templates
├── deployment/         docker-compose.yml, .env.example, deploy.sh, restore.sh
├── gitlab-deploy/      .gitlab-ci.yml, sync.sh, setup-variables.sh
└── docs/captures/      captures du rapport et script qui les produit (capture.py)
```

03 Qualité : tests et SonarQube

PYRAMIDE DE TESTS

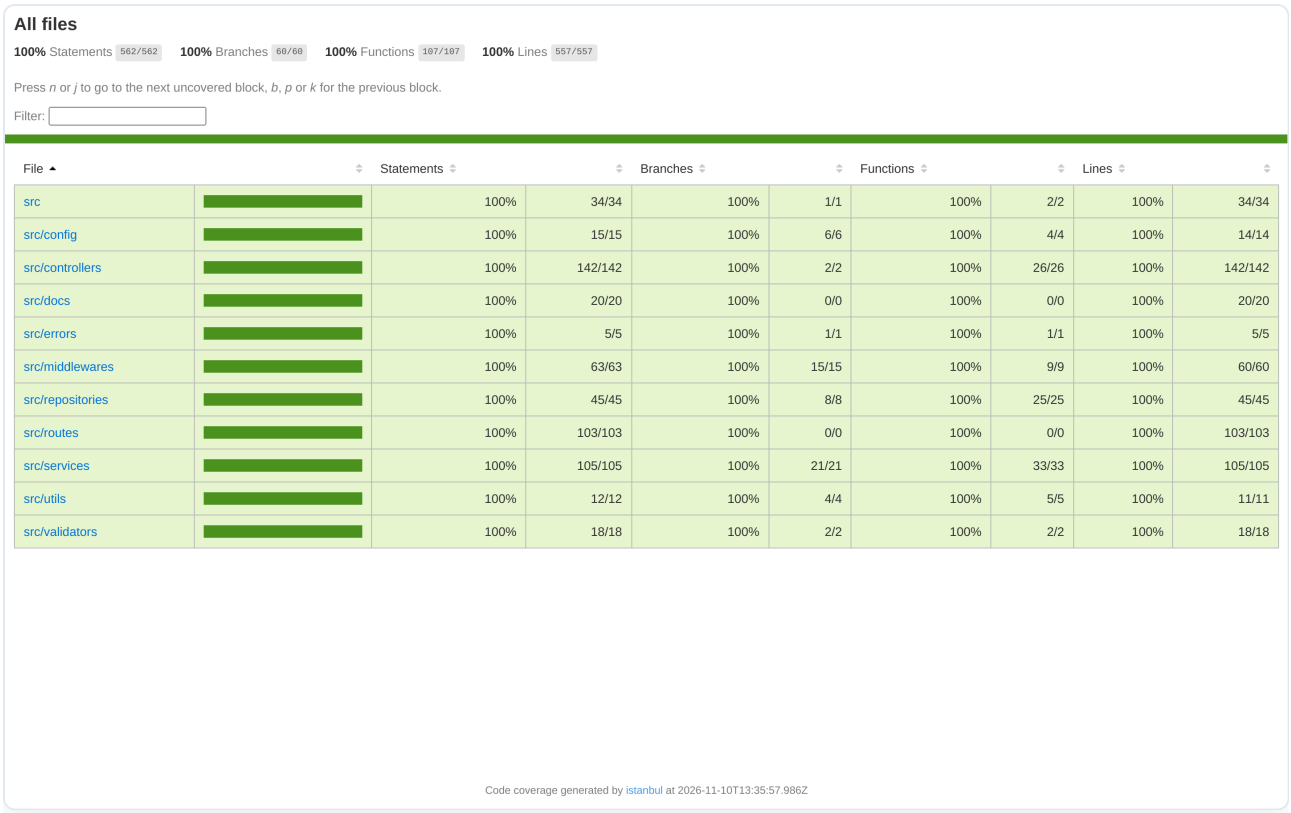
NIVEAU	TESTS	CE QUI EST RÉEL	CE QUI EST SIMULÉ
Unitaires	244 (74 %)	une classe ou une fonction	toutes ses dépendances
Intégration	64 (19 %)	toute l'API HTTP : routes, auth, RBAC, validation, erreurs	Prisma uniquement
E2E	22 (7 %)	serveur + PostgreSQL réels : parcours de recrutement complet	rien

Fonctionnalités couvertes : authentification (JWT), candidats, offres, candidatures, contrôle d'accès ADMIN/RECRUITER, pagination, validation, codes d'erreur.



▸ Résultat des tests

La couverture atteint **100 % sans base de données**, donc la CI n'a pas besoin de PostgreSQL. Elle échoue sous 95 %. Les rapports `lcov.info` (SonarQube), `sonar-report.xml` (SonarQube) et `junit.xml` (Jenkins) sont produits par `npm run test:ci`.

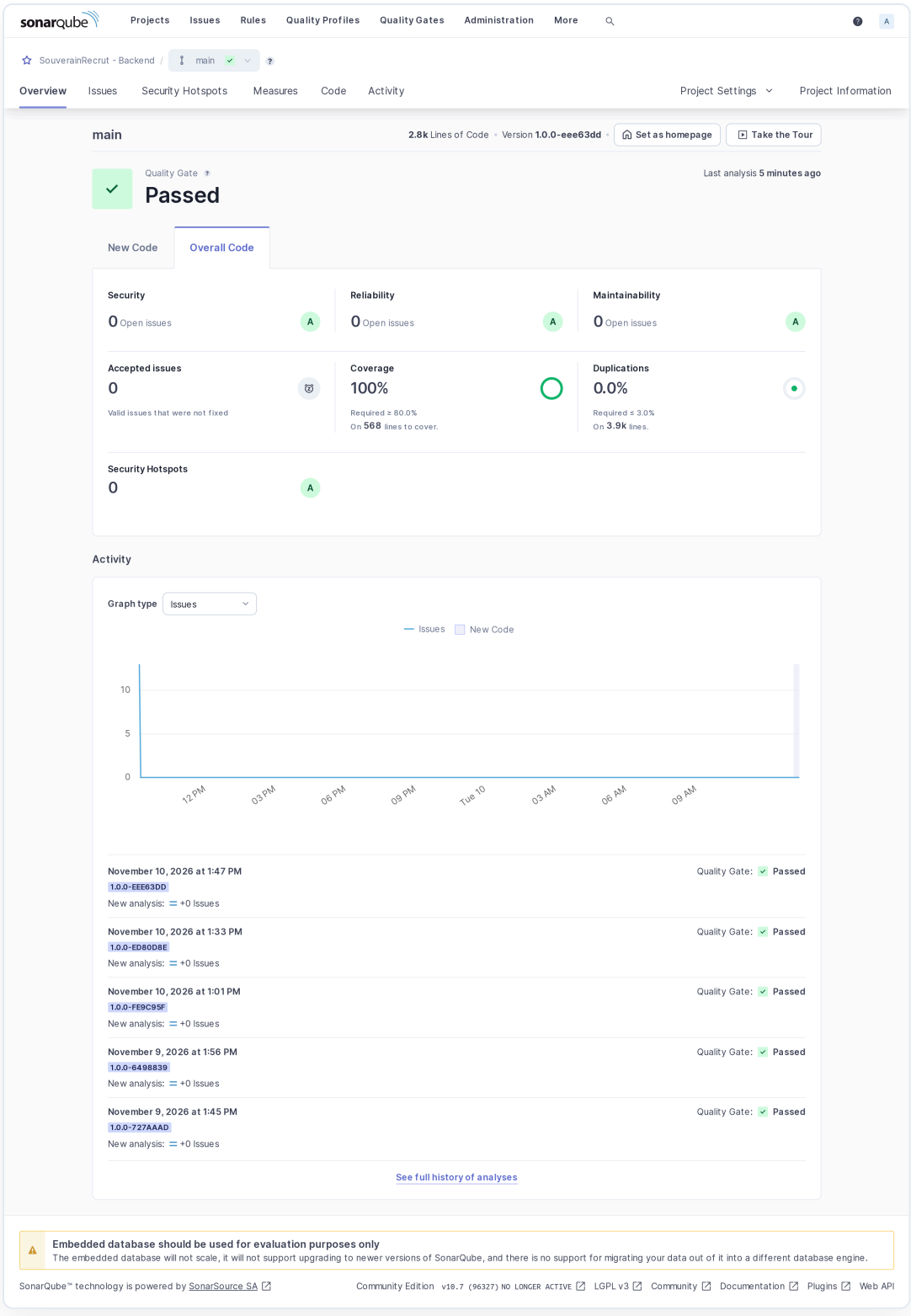


▸ Rapport de couverture

QUALITY GATE « SOUVERAINRECRUT »

CONDITION	SEUIL
Fiabilité, sécurité, maintenabilité	note A
Security hotspots revus	100 %
Couverture	≥ 80 %
Duplication	≤ 3 %

Première analyse : 13 code smells et 1 hotspot (`X-Powered-By`), tous corrigés. Le gate a aussi été **vérifié en échec** : sans rapport de couverture, il passe à FAILED et la pipeline s'arrête.

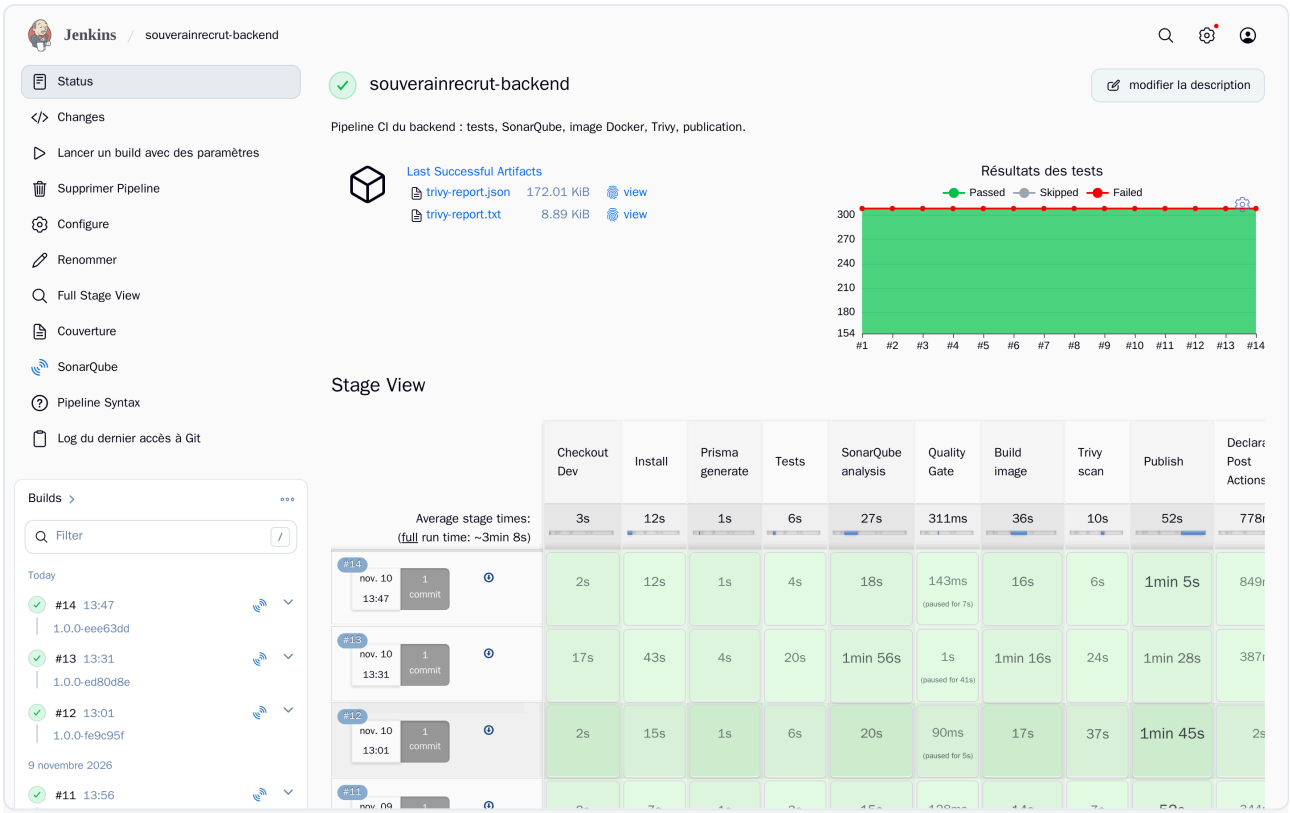


▸ Tableau de bord SonarQube

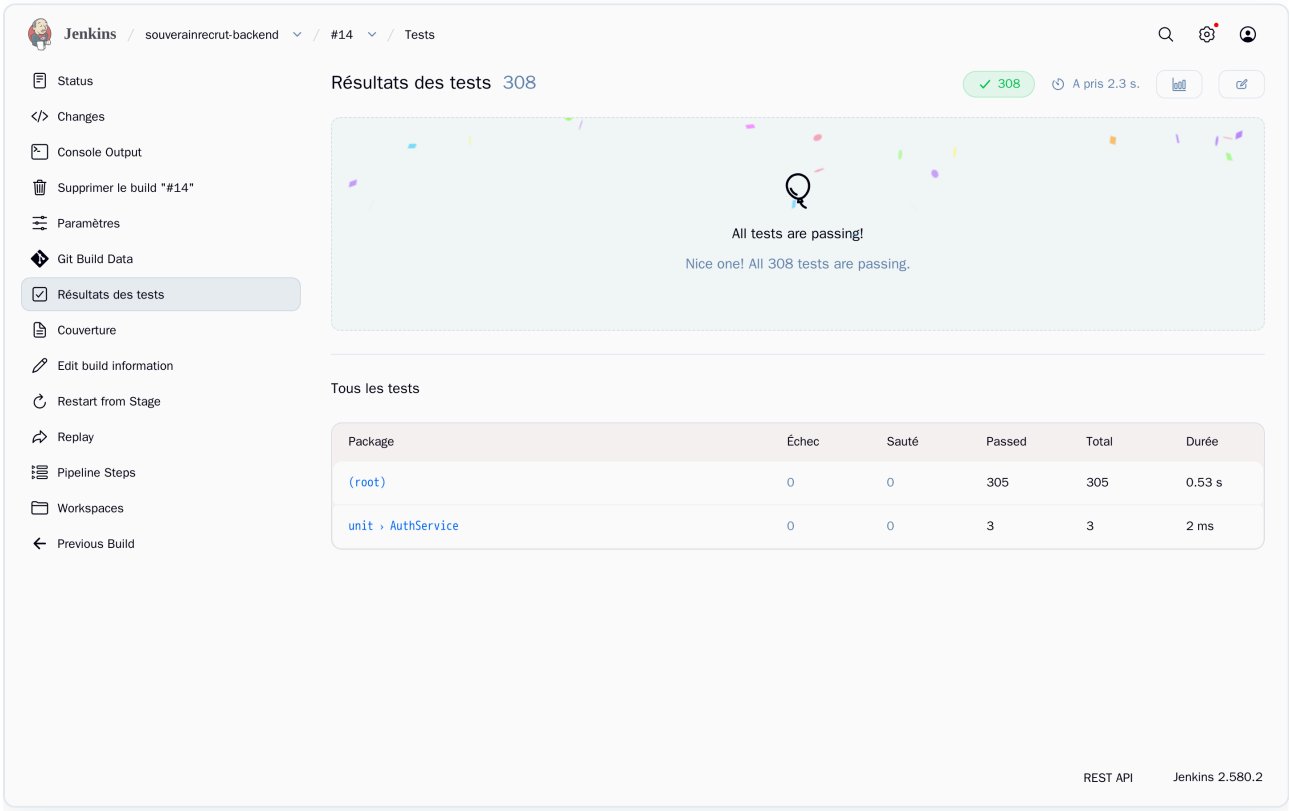
04 Intégration continue : Jenkins

Jenkins est entièrement décrit par du code : `ci/jenkins/Dockerfile` (Node 22, CLI Docker + buildx, Trivy vérifié par SHA-256), `plugins.txt` et `casc.yaml` (administrateur, serveur SonarQube, credentials, job). Le job scrute la branche `Dev`.

ÉTAPE	RÔLE	BLOQUANTE
Checkout Dev	Récupère le commit, calcule le tag `<version>-<commit>`	—
Install, Prisma generate	`npm ci` (lockfile), client Prisma	oui
Tests	`typecheck` + `test:ci`, publication JUnit et couverture HTML	oui
SonarQube analysis	Analyse avec version = tag	oui
Quality Gate	Attend le verdict par webhook	**oui**
Build image	`docker buildx build --platform linux/amd64`	oui
Trivy scan	Rapport complet archivé, puis contrôle bloquant	**oui**
Publish	`docker login` (credential Jenkins), push `<version>-<commit>` et `dev`	uniquement si tout est vert



▸ Pipeline Jenkins



▸ Tests publiés dans Jenkins

05 Image Docker et sécurité

EXIGENCE	MISE EN ŒUVRE (`BACKEND/DOCKERFILE`)
Reproductible	Image de base figée par version **et** digest, `npm ci` sur le lockfile
Minimale	Build multi-étapes, dépendances de production seulement, `npm` / `npx` retirés de l'image finale, `.dockerignore` strict
Sans secret	`.env` exclu du contexte ; secrets fournis au lancement ; Trivy recherche les secrets
Sûre	Utilisateur `node` (non root), `HEALTHCHECK`
Traçable	Tag `<version>-<commit>` + labels OCI `version` et `revision` (commit complet)
Compatible	Construite pour `linux/amd64`, l'architecture du serveur (Jenkins tourne sur un Mac arm64)

SEUILS TRIVY

CATÉGORIE	DÉCISION
HIGH / CRITICAL avec correctif	**bloquant**
Secret détecté	**bloquant**
MEDIUM / LOW, ou sans correctif	rapport archivé, non bloquant
Exception	`backend/.trivyignore.yaml` : justifiée, limitée à un paquet et une version, **date d'expiration**

Le premier scan a bloqué la pipeline sur 26 vulnérabilités HIGH. Traitement : `npm` retiré de l'image, dépendances transitives mises à jour (versions publiées depuis au moins 2 semaines), Prisma 6.19.3. Une seule exception subsiste : `deepmerge-ts` (CVE-2026-40345). Elle n'est chargée que par la CLI Prisma pour lire sa configuration, jamais sur une entrée utilisateur ; le correctif n'existe qu'en version 8, que Prisma 6 ne supporte pas. Elle expire le 09/02/2027.

trivy image — synthèse du rapport archivé par Jenkins (image 1.0.0-727aaad)

```
$ trivy image --scanners vuln,secret souverainrecrut-backend:1.0.0-727aaad
  CIBLE                              CRIT.  HIGH  MED.   LOW
  Paquets système (Alpine 3.24)         0     0     1     0
  Dépendances Node.js (node_modules)    0     1     4     0
  TOTAL                                 0     1     5     0
  Secrets détectés dans l'image : 0

$ trivy image --severity HIGH,CRITICAL --ignore-unfixed --ignorefile .trivyignore.yaml --exit-code 1
  HIGH/CRITICAL corrigeables : deepmerge-ts@7.1.5 CVE-2026-40345
  → CVE-2026-40345 couverte par l'exception justifiée (expire le 2027-02-09)
  → 0 vulnérabilité bloquante : exit 0, publication autorisée
```

▸ Synthèse Trivy

Tags publiés sur Docker Hub (API hub.docker.com)

```
$ GET /v2/repositories/martinbellot/souverainrecrut-backend/tags
  TAG              ARCH     PUBLIÉ (UTC)        DIGEST
  dev              amd64    2026-11-10 13:38    sha256:80756fbd5f7d
  1.0.0-ed80d8e    amd64    2026-11-10 13:38    sha256:80756fbd5f7d
  1.0.0-fe9c95f    amd64    2026-11-10 13:04    sha256:5cd09241bbdc
  1.0.0-6498839    amd64    2026-11-09 13:57    sha256:a037a781b2b7
  1.0.0-727aaad    amd64    2026-11-09 13:47    sha256:68d6aea0d64a
  1.0.0-e34613f    arm64    2026-11-09 11:24    sha256:828027aaf333
```

▸ Tags publiés sur Docker Hub

> La page web de Docker Hub bloque les navigateurs automatisés (Cloudflare) : la liste ci-dessus est construite à partir de l'API publique de Docker Hub. On y voit `1.0.0-e34613f` en arm64 : c'est la publication antérieure au correctif d'architecture (§ 11).

06 Infrastructure : Terraform et Ansible

Terraform (`terraform/`). Sans ESXi/vCenter, le provider vSphere est inutilisable. Terraform gère donc les ressources réellement disponibles autour du VPS :

RESSOURCE	RÔLE
`tls_private_key`	Paire de clés SSH dédiée au déploiement (clé privée hors dépôt, dans `.secrets/`)
`null_resource` + `remote-exec`	Contrôle du serveur en lecture seule : OS, architecture, IP, accès Internet
`local_file`	Génère `ansible/inventory.ini` et les variables Ansible

terraform apply — vérification du serveur et génération de l'inventaire Ansible

```
$ terraform apply
  OS           : Debian GNU/Linux 13 (trixie)
  Architecture: amd64
  IP           : 148.230.115.48
  Internet     : OK
Apply complete! Resources: 1 added, 0 changed, 1 destroyed.
ansible_command = "cd ansible && ansible-playbook -i inventory.ini playbook.yml"
api_url = "http://148.230.115.48:4000/v1"
deploy_private_key_path = "./terraform/.secrets/deploy_ed25519"
deploy_user = "deploy"
server_ip = "148.230.115.48"
```

▸ terraform apply

Ansible (`ansible/playbook.yml`), `ansible-playbook -i inventory.ini playbook.yml` :

ÉTAPE	DÉTAIL
Système	Mise à jour APT ; `needrestart` en mode liste (aucun service redémarré) ; redémarrage signalé, jamais forcé
Docker	Docker et Compose existants conservés ; sinon, installation de Docker CE depuis le dépôt officiel. Service activé et vérifié
Accès	Utilisateur `deploy` (groupe docker) avec la clé générée par Terraform
Application	`/opt/souverainrecrut` (0750), fichiers de `deployment/`, secrets initiaux générés **sur le serveur** (0600), compte admin
Réseau	ufw : port 4000 ouvert
Contrôle	Conteneurs, `/v1/ready` depuis le serveur, `/v1/health` depuis l'extérieur

Le playbook est **idempotent** : un second passage ne change rien (`changed=0`).

ansible-playbook — configuration idempotente du serveur (2e passage)

```
$ ansible-playbook -i inventory.ini playbook.yml

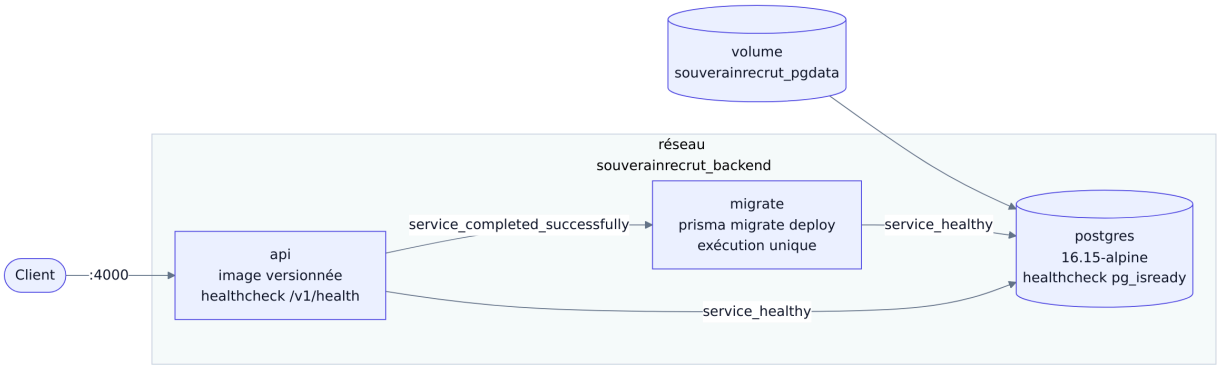
  ok        Vérifier que le système est supporté
  ok        Mettre à jour le cache et les paquets du système
  ok        Installer les dépendances de base
  ok        Vérifier si un redémarrage est nécessaire
  ok        Détecter une installation Docker existante
  skipping  Créer le répertoire des clés APT
  skipping  Télécharger la clé GPG de Docker
  skipping  Ajouter le dépôt APT de Docker
  skipping  Installer Docker Engine et le plugin Compose
  ok        Activer et démarrer le service Docker
  ok        Vérifier Docker Engine et le plugin Compose
  ok        Créer l'utilisateur de déploiement (membre du groupe docker)
  ok        Autoriser la clé SSH de déploiement (générée par Terraform)
  ok        Créer le répertoire de déploiement
  ok        Générer la configuration et les secrets initiaux (.env, admin.env)
  ok        Installer les fichiers de déploiement (deployment/)
  ok        Installer le script d'amorçage du compte administrateur
  ok        Ouvrir le port de l'API dans le pare-feu
  ok        Lire le tag actuellement déployé
  skipping  Déployer l'image Backend (premier déploiement ou tag imposé)
  ok        Créer le compte administrateur initial
  ok        Vérifier les conteneurs de la stack
  ok        Vérifier l'API et sa connexion à PostgreSQL (depuis le serveur)
  ok        Vérifier l'accès public à l'API (depuis le poste Ansible)

PLAY RECAP
  souverainrecrut-vps : ok=21 changed=0 unreachable=0 failed=0 skipped=5 rescued=0 ignored=0
```

▸ ansible-playbook

07 Déploiement : Docker Compose et GitLab CI

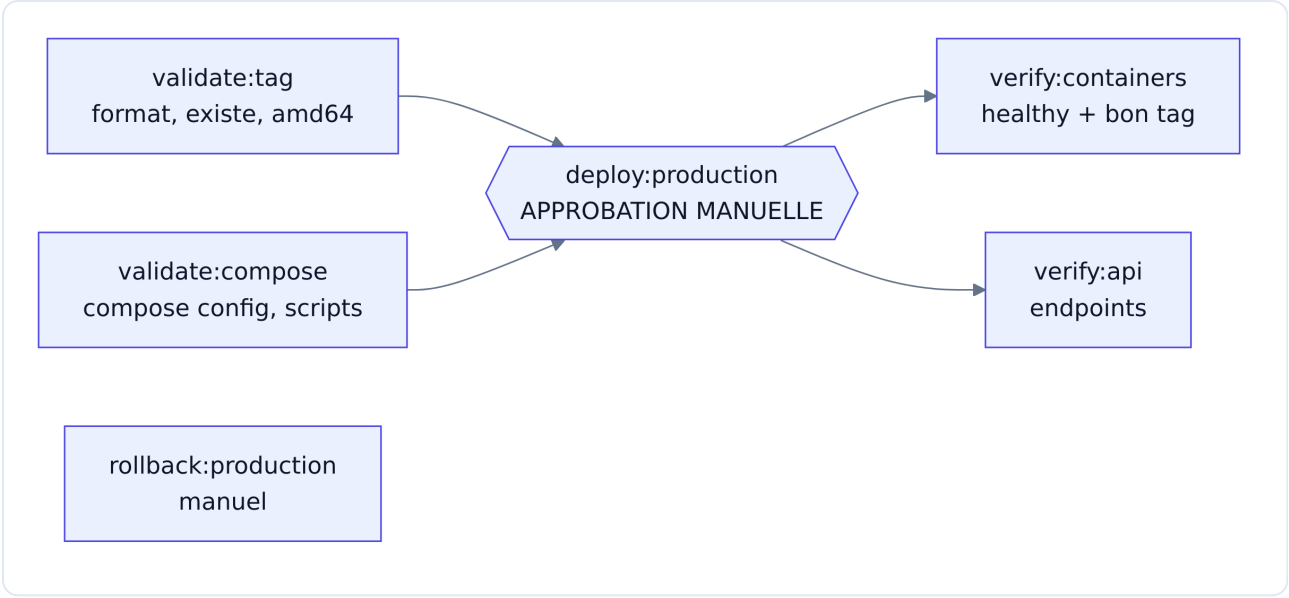
`deployment/docker-compose.yml`



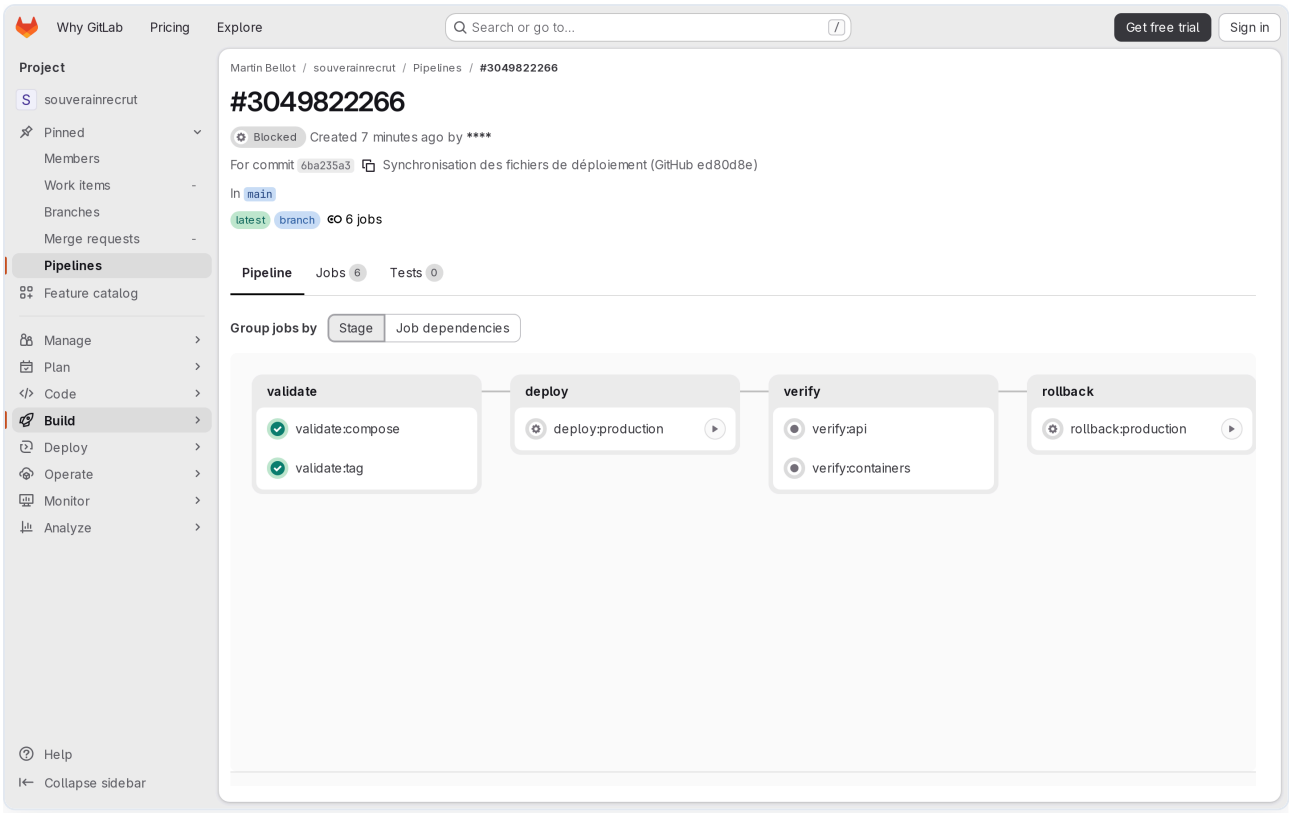
EXIGENCE 5.2	MISE EN ŒUVRE
Image versionnée	`${BACKEND_IMAGE}:${IMAGE_TAG}`, tag dans `release.env` écrit par `deploy.sh`
Variables API et Prisma	`DATABASE_URL`, `JWT_SECRET`, `PORT`… depuis `.env` (modèle : `.env.example`)
Persistance et réseau	Volume nommé `pgdata` ; réseau dédié ; PostgreSQL **non exposé**
Redémarrage et santé	`restart: unless-stopped` ; healthchecks PostgreSQL et API ; rotation des logs
Migrations maîtrisées	Service `migrate` exécuté avant chaque démarrage de l'API ; `migrate deploy` n'applique que les migrations versionnées (jamais de reset) ; **sauvegarde `pg_dump` automatique avant**

`deployment/deploy.sh` : sauvegarde → `pull` → `up` (migrations puis API) → contrôle de santé → historique. En cas d'échec, il revient automatiquement à la version précédente.

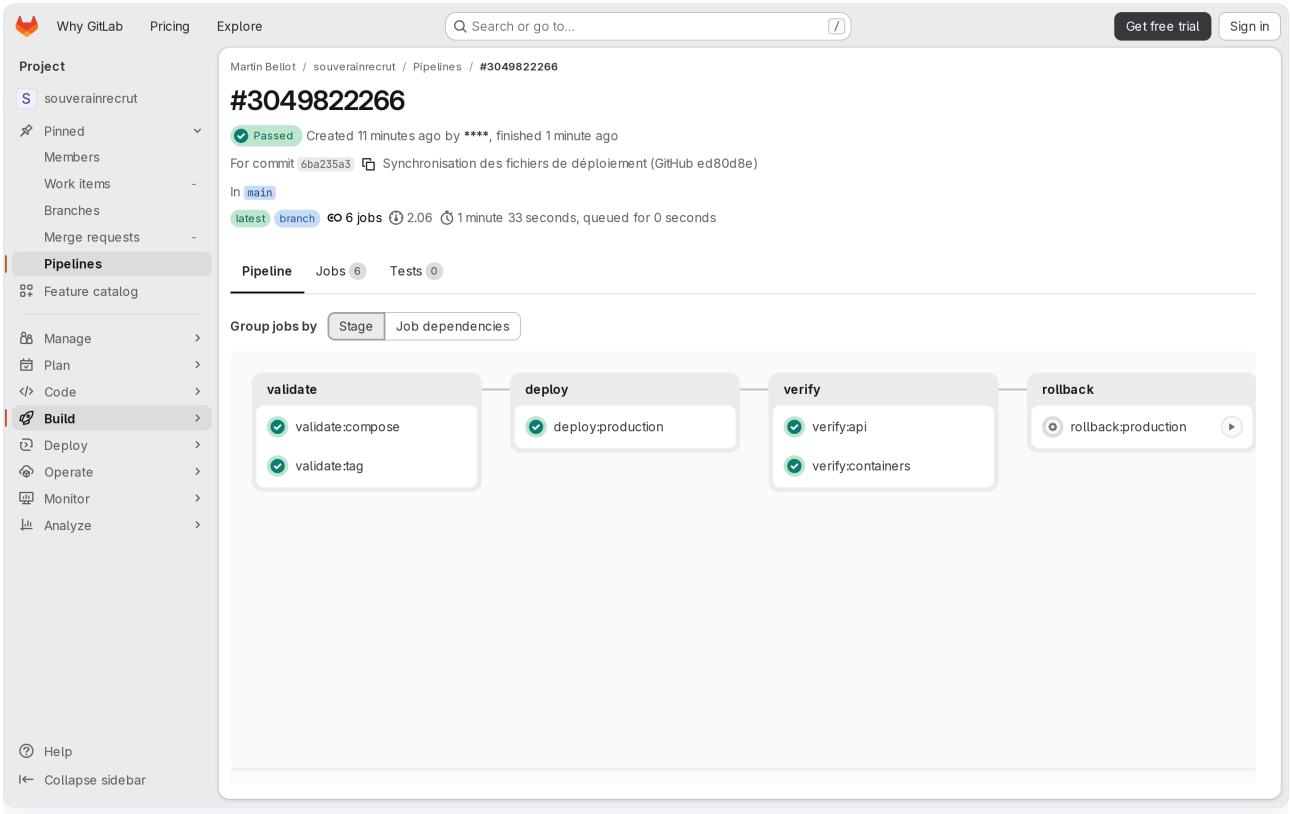
PIPELINE GITLAB



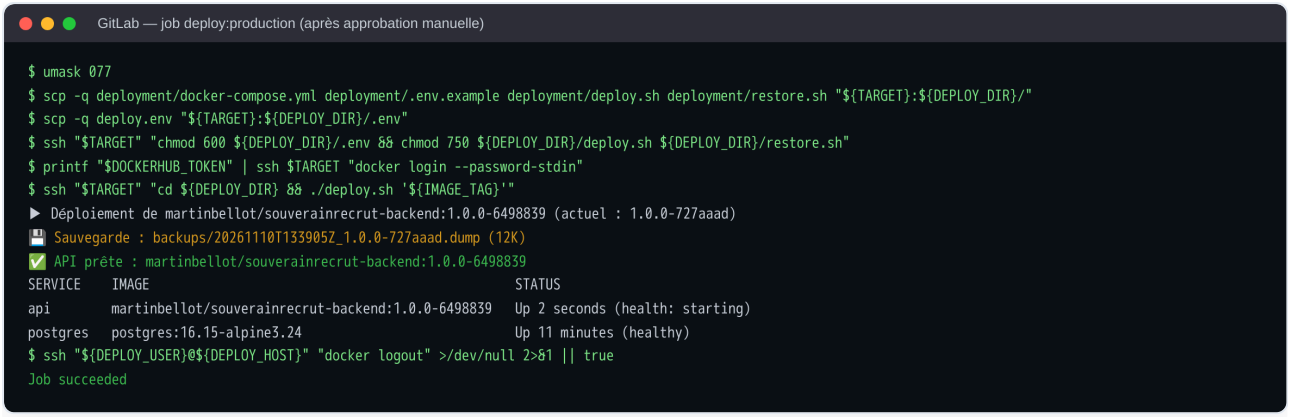
EXIGENCE 5.3	MISE EN ŒUVRE
Valider le tag	Format `<version>-<commit>`, présence sur Docker Hub, architecture amd64
Connexion SSH	Agent SSH + clé (variable *File*), hôte vérifié par `known_hosts`
Transmettre fichiers et configuration	`scp` de `deployment/` et du `.env` généré depuis les variables protégées
Récupérer l'image	`docker login` (token par stdin) puis `docker compose pull`
Démarrer ou mettre à jour	`./deploy.sh <tag>`
Vérifier	Conteneurs `healthy` sur le bon tag ; `/health`, `/ready`, `/info`, `/api-docs`, 401 sans token, 400 sur login invalide
Approbation	Job `deploy:production` en `when: manual` : le pipeline reste bloqué jusqu'à validation



▸ Pipeline en attente d'approbation



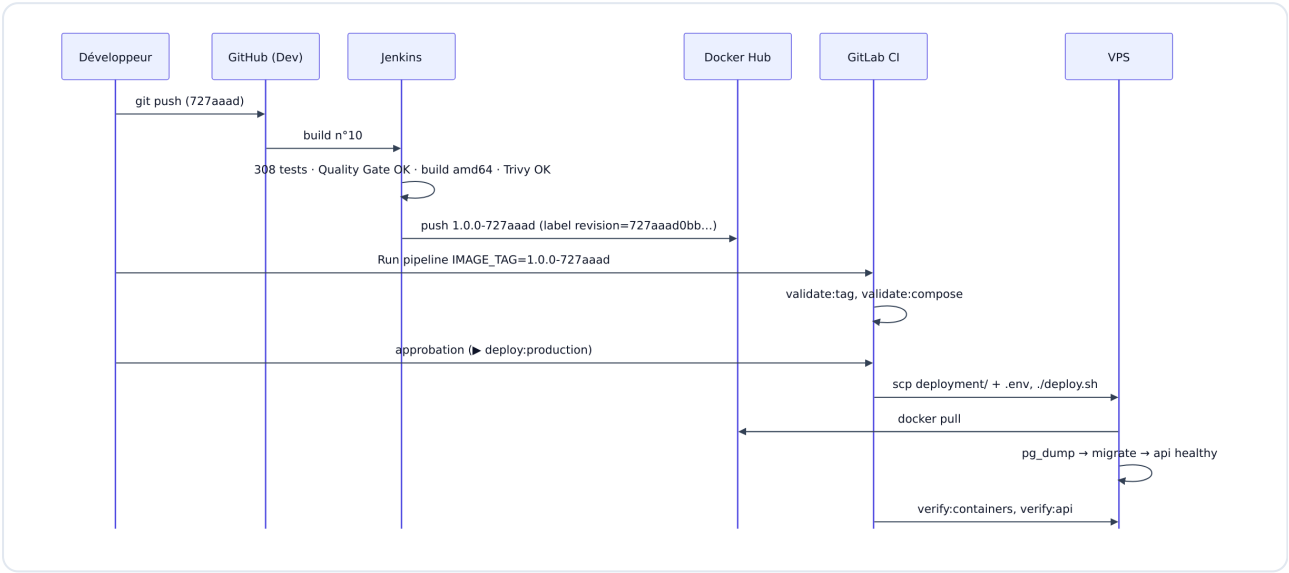
▸ Pipeline après approbation, déploiement et vérifications



▸ Job deploy:production

08 Vérification, rollback et données

CHEMINEMENT COMPLET D'UNE MODIFICATION



Sur le serveur, la version en service se retrouve depuis le conteneur lui-même : `docker inspect souverainrecrut-api-1` donnait `revision=727aaad0bb150b3c2fb76200c970e73a5d4061e7`. **Version en production à la remise : `1.0.0-eee63dd`**, déployée par le même pipeline GitLab (historique complet dans `/opt/souverainrecrut/release.history`).

VÉRIFICATIONS RÉALISÉES SUR LE SERVEUR

VÉRIFICATION	RÉSULTAT
Endpoints principaux	`/health` UP, `/ready` READY + base CONNECTED, `/info`, Swagger, login admin, création et suppression de candidat, 401 sans token
Communication API ↔ PostgreSQL	`/ready` exécute `SELECT 1` ; écriture et lecture via l'API
Persistance	Donnée témoin conservée après `docker compose down` puis redémarrage
Sauvegarde et restauration	Donnée supprimée, puis restaurée depuis la sauvegarde automatique
Rollback	`1.0.0-6498839` → `1.0.0-727aaad` par le job GitLab ; donnée témoin intacte
Retour automatique	Tag inexistant : aucun changement. Image incompatible : retour automatique à la version précédente

Persistance PostgreSQL — destruction et recréation de la stack (volume conservé)

```
$ curl -X POST /v1/candidates   # donnée témoin
→ candidat créé : cmwaq427u0000lh01a8pqn1jo

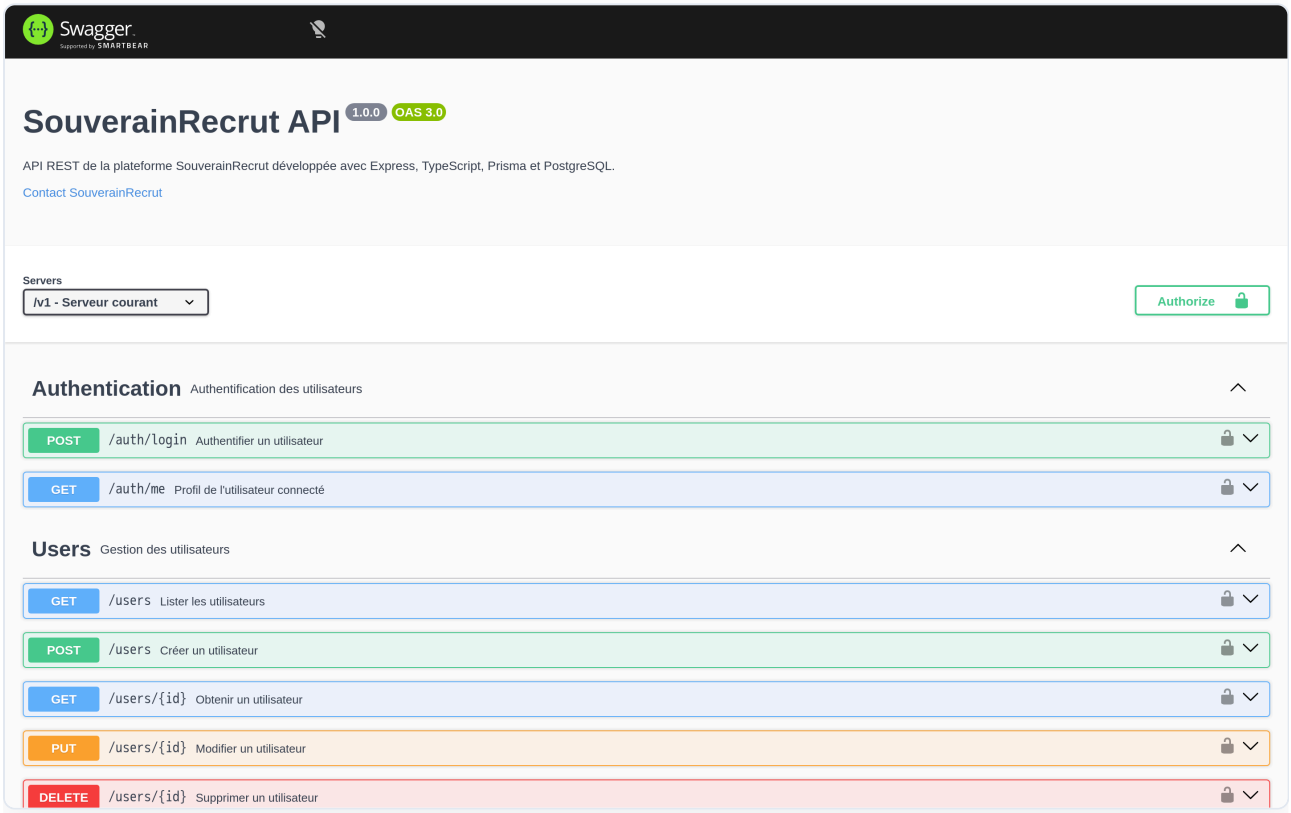
$ docker compose down
 Container souverainrecrut-api-1       Removed
 Container souverainrecrut-migrate-1   Removed
 Container souverainrecrut-postgres-1  Removed
 Network souverainrecrut_backend       Removed

$ docker volume ls --filter name=souverainrecrut
souverainrecrut_pgdata

$ ./deploy.sh 1.0.0-727aaad
▶ Déploiement de martinbellot/souverainrecrut-backend:1.0.0-727aaad (actuel : 1.0.0-727aaad)
💾 Sauvegarde : backups/20261110T132813Z_1.0.0-727aaad.dump (12K)
✅ API prête : martinbellot/souverainrecrut-backend:1.0.0-727aaad

$ curl /v1/candidates/cmwaq427u0000lh01a8pqn1jo
→ témoin toujours présent : Témoin Persistance <temoin-persistance@example.com>
```

▸ Persistance

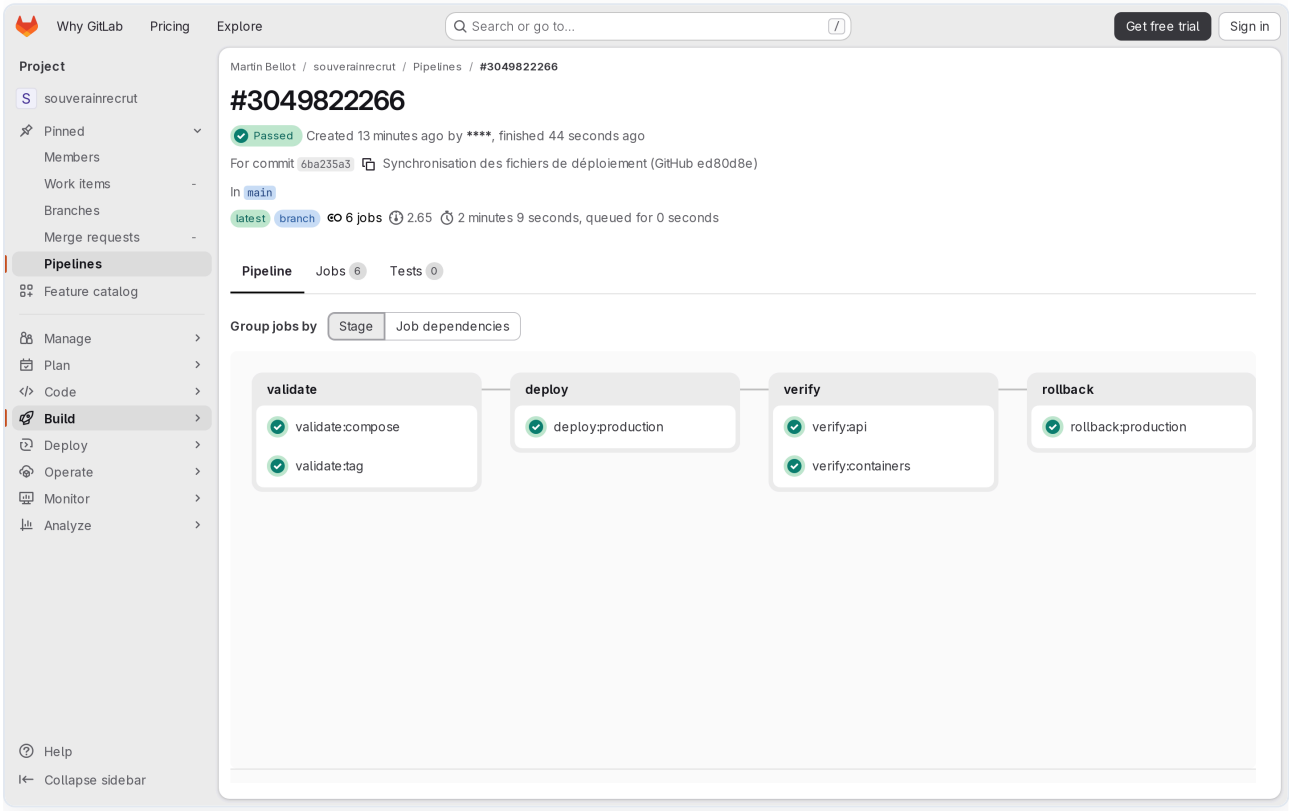


▸ Swagger UI en production

PROCÉDURE DE ROLLBACK

SITUATION	ACTION	DONNÉES
Le déploiement échoue (image, migration, santé)	**Automatique** : `deploy.sh` remet la version précédente	inchangées

SITUATION	ACTION	DONNÉES
Défaut fonctionnel constaté après un déploiement réussi	GitLab : job **`rollback:production`**, ou *Run pipeline* avec `IMAGE_TAG` = version précédente (liste dans `release.history` ou sur Docker Hub)	inchangées
La migration de la nouvelle version a cassé le schéma ou les données	Rollback applicatif, **puis** `./restore.sh <sauvegarde> --yes`	données écrites après la sauvegarde **perdues**

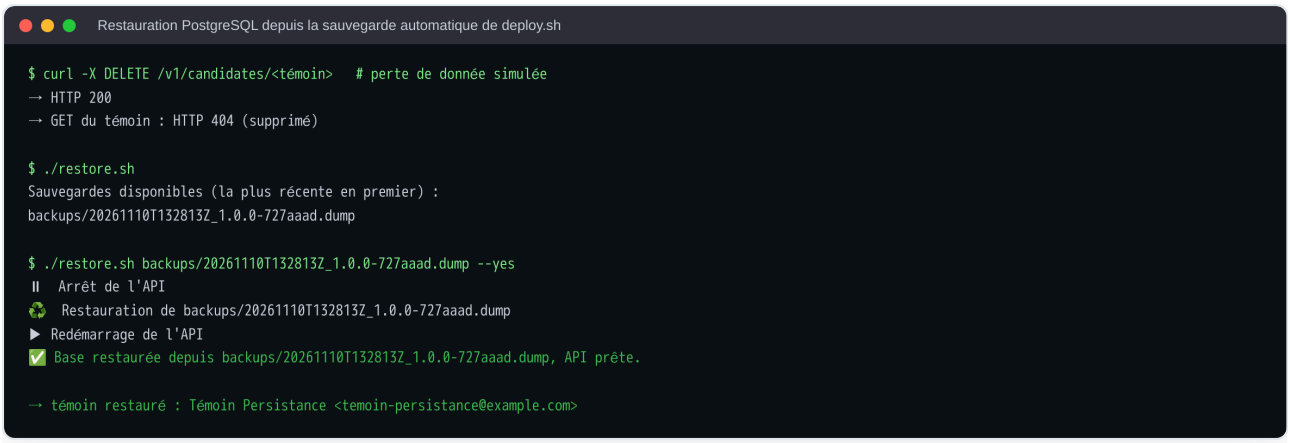


▸ Rollback par GitLab

```
GitLab — job rollback:production (scénario maîtrisé)

$ ssh $TARGET "cd /opt/souverainrecrut && ./deploy.sh --rollback"
↩ Rollback demandé : 1.0.0-6498839 → 1.0.0-727aaad
▶ Déploiement de martinbellot/souverainrecrut-backend:1.0.0-727aaad (actuel : 1.0.0-6498839)
💾 Sauvegarde : backups/20261110T134154Z_1.0.0-6498839.dump (12K)
✅ API prête : martinbellot/souverainrecrut-backend:1.0.0-727aaad
SERVICE    IMAGE                                          STATUS
api        martinbellot/souverainrecrut-backend:1.0.0-727aaad   Up 2 seconds (health: starting)
postgres   postgres:16.15-alpine3.24                      Up 14 minutes (healthy)
Job succeeded
```

▸ Job rollback:production



▸ Restauration de la base

Migrations et réversibilité. Un rollback applicatif ne défait pas une migration Prisma : l'ancienne version de l'API tourne alors sur le nouveau schéma. Règles retenues :

- **Migrations additives (expand / contract)** : ajouter une colonne ou une table d'abord, compatible avec l'ancien code ; ne supprimer ou renommer que dans une version ultérieure, quand plus aucune version déployée n'en dépend. Le rollback reste alors sûr.
- **Sauvegarde avant chaque déploiement** (`deploy.sh`, 7 conservées) : c'est le filet de sécurité quand une migration n'est pas réversible.
- `migrate deploy` uniquement en production, jamais `migrate dev` ni `reset`.

09 Gestion des secrets

SECRET	STOCKÉ DANS	UTILISÉ PAR
Token Docker Hub (publication)	Credential Jenkins (JCasC, depuis `ci/.env` non versionné)	étape Publish
Token SonarQube	Credential Jenkins	analyse
Token Docker Hub (pull)	Variable GitLab **protégée, masquée et cachée**	`deploy:production`
Clé SSH de déploiement	État Terraform local + variable GitLab de type *File*	jobs SSH
`POSTGRES_PASSWORD`, `JWT_SECRET`	Variables GitLab masquées et cachées → `.env` 0600 sur le serveur	stack
Mot de passe admin de l'API	`admin.env` 0600 sur le serveur	exploitation



▸ Variables GitLab

Contrôles effectués : aucun motif de secret ni aucune valeur réelle dans les 13 commits de l'historique Git ; 0 occurrence des 4 secrets dans les journaux des jobs GitLab ; pas de `set -x` ; token transmis par stdin ; `.env` créé en 0600 puis supprimé du runner.

10 Conformité aux exigences

EXIGENCE	ATTENDU	PREUVES
Reproductibilité	Build, configuration et déploiement décrits et rejouables	`Jenkinsfile`, JCasC, Dockerfile figé par digest, Terraform, playbook idempotent (`changed=0`), `deployment/`, `.gitlab-ci.yml`, `capture.py` — § 4 à 7
Qualité logicielle	Tests exécutés, couverture et SonarQube consultables	330 tests, 100 % de couverture, rapport HTML dans Jenkins, Quality Gate PASSED — § 3
Sécurité	Secrets protégés, image analysée par Trivy	Variables protégées, masquées et cachées, credentials Jenkins, historique et journaux vérifiés ; Trivy bloquant avec seuils documentés — § 5, § 9
Traçabilité	Tag explicite relié à une version du code	`<version>-<commit>`, labels OCI `revision`, version SonarQube = tag, `release.history` — § 5, § 8
Déploiement	Le serveur récupère l'image publiée, l'API démarre avec PostgreSQL	`docker pull` depuis Docker Hub (aucun build sur le serveur), migrations puis API `healthy` — § 7
Continuité	Rollback documenté	Trois niveaux de rollback testés + sauvegarde et restauration — § 8
Documentation	Choix, prérequis et procédures expliqués	Ce rapport, `README.md`, `gitlab-deploy/README.md`, commentaires des fichiers — § 13

11 Défauts détectés et corrigés

Les tests et les déploiements réels ont révélé des défauts. Chacun a été corrigé, puis revérifié.

ORIGINE	DÉFAUT	CORRECTION
Base de données	La migration ne correspondait pas au schéma : pas de suppression en cascade, pas d'unicité des candidatures	Migration corrective
Sécurité API	Hash des mots de passe renvoyé par `/users` ; mot de passe enregistré **en clair** à la mise à jour	Sélection sans mot de passe ; hachage systématique
Robustesse API	Erreurs 500 sur données invalides (Prisma, JSON, login, pagination)	Codes 400/404/409/413 et validation Zod
Configuration	Démarrage possible sans `JWT_SECRET` ; port exposé ≠ port d'écoute	Arrêt au démarrage ; `PORT=4000`
SonarQube	13 code smells, 1 hotspot, nom de projet mal encodé	Corrigés
Trivy	26 vulnérabilités HIGH (dont `proxy-addr`, `npm` embarqué)	Mises à jour, `npm` retiré, exception datée
Architecture	Image publiée en **arm64** (Jenkins sur Mac), alors que le serveur est en amd64	Build `buildx --platform linux/amd64` + contrôle dans GitLab
Déploiement	Retour arrière inopérant (`set -e`) ; échec après un `compose down` (variable manquante, base arrêtée pendant la sauvegarde)	Script corrigé et revérifié
GitLab	YAML invalide (`:` lu comme une clé)	Détecté par le lint, corrigé
CI	Avertissement Jest en CI (13 workers sur 14 cœurs)	`--maxWorkers=2`

> Deux de ces tests ont coupé l'API SouverainRecrut pendant quelques minutes, ce qui a permis de corriger `deploy.sh`. Les autres services du serveur n'ont jamais été touchés.

12 Limites et pistes

LIMITE	PISTE
Jenkins et SonarQube tournent sur le poste de développement	Les héberger sur un serveur CI dédié, avec un webhook GitHub à la place de la scrutation
SonarQube 10.7 n'est plus maintenu et utilise sa base embarquée (réservée à l'évaluation)	Version LTA courante avec une base PostgreSQL dédiée
Coupure d'environ 30 s à 1 min pendant un déploiement ou un rollback	Déploiement blue/green derrière un reverse proxy
API en HTTP sur le port 4000	HTTPS via un reverse proxy et un nom de domaine
`deploy` est membre du groupe `docker`, ce qui équivaut à des droits root	Docker rootless, ou commande SSH forcée vers `deploy.sh`
Exception Trivy `deepmerge-ts`	Migrer vers Prisma 7
Sauvegardes conservées sur le serveur lui-même	Les copier hors du serveur (stockage objet)

13 Procédures de référence

Prérequis : Docker, Node 22, Terraform ≥ 1.6, Ansible ≥ 2.16, un accès SSH root au serveur (agent SSH), des comptes GitHub, GitLab (vérifié) et Docker Hub.

OBJECTIF	COMMANDES
Lancer Jenkins + SonarQube	`cp ci/.env.example ci/.env` (renseigner) · `docker compose -f docker-compose.ci.yml up -d --build` · `SONAR_HOST_URL=… SONAR_ADMIN_TOKEN=… ci/sonar/setup-quality-gate.sh`
Tester en local	`cd backend && npm ci && npm test` · E2E : `npm run test:db:up && npm run test:e2e`
Provisionner	`cd terraform && terraform init && terraform plan && terraform apply`
Configurer le serveur	`cd ansible && ansible-playbook -i inventory.ini playbook.yml -e image_tag=<tag>`
Préparer GitLab	`./gitlab-deploy/setup-variables.sh <tag>` · `./gitlab-deploy/sync.sh`
Déployer	GitLab → *Run pipeline* (`IMAGE_TAG=<tag>`) → ▶ `deploy:production`
Revenir en arrière	GitLab → ▶ `rollback:production` (ou *Run pipeline* avec l'ancien tag)
Restaurer la base	Sur le serveur : `./restore.sh` (liste) puis `./restore.sh backups/<fichier>.dump --yes`
Observer	`docker compose --env-file .env --env-file release.env ps` · `… logs -f api`
Refaire les captures	voir l'en-tête de `docs/captures/capture.py`
Régénérer `RAPPORT.pdf`	`./docs/pdf/build.sh` (Markdown → HTML → Chromium, schémas Mermaid en PNG)